

NEXIS

University of Dhaka
Institute of Information Technology (IIT)

Project Report On

“Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery”

SUBMITTED BY

Ratul Sikder

Exam Roll: 2506102

Supervised By: _____

Institute of Information Technology (IIT), University of Dhaka

Date: 11th August, 2026 — Condensed Edition

Declaration. This is to declare that this project is my original work. No part of this work has been submitted elsewhere, partially or entirely, for the award of any other degree or diploma.

Signature Page.

Project	:	Nexis: A Multi-Agent Autonomous Engineering Platform with Closed-Loop Fault Recovery
Student Name	:	Ratul Sikder
Exam Roll	:	2506102
Institute	:	Institute of Information Technology (IIT), University of Dhaka
Date of Submission	:	11th August, 2026
Supervised By	:	_____
Supervisor's Approval	:	_____ _____

Abstract

Modern software engineering teams spend an estimated 40–60% of their time on maintenance, debugging, and manual incident response rather than building new capability. Existing AI-assisted developer tools — code completion, automated test generation, deployment dashboards — address individual tasks in isolation; none model the collaborative structure of a real engineering team, and none close the loop between detecting a production failure and autonomously repairing it. **Nexis** addresses both gaps at once. It models a software engineering organisation as nine role-specialised autonomous agents across two cooperating layers — a five-agent Execution Team (Architect, Backend, Quality Assurance (QA), DevOps, Data Engineer) and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) — durably orchestrated by Temporal and gated by a severity-routed human-in-the-loop Approval Gate that supports Approve, Reject, and Modify (Reinforcement Learning from Human Feedback (RLHF)) decisions. This report documents the complete, working system: a multi-tenant control plane (Go (Golang), PostgreSQL with Row-Level Security (RLS), Neo4j, Redis), a Docker shadow-validation sandbox, a real GitHub-App-mediated GitOps deployment path, a from-scratch Docker preview-deploy engine, and a Next.js console spanning authentication, Role-Based Access Control (RBAC), incident observability, billing, and multi-provider integrations. Two live end-to-end recovery runs against a local Large Language Model (LLM) are reported verbatim, alongside a live, verified proof of the preview-deploy engine. What is deliberately out of scope — a fully credentialed production deployment to the registered `nexis.sh` domain, and the human-subject NASA Task Load Index (NASA-TLX) study — is stated plainly rather than implied.

Keywords: Multi-agent systems, autonomous software engineering, self-healing systems, closed-loop fault recovery, root-cause analysis, LLM-guided program repair, DevOps automation, Temporal workflow orchestration, human-in-the-loop AI.

This is the condensed edition of the full project report. The complete 107-page edition with exhaustive feature-by-feature screenshots is available as a companion document.

Table of Contents

Chapter 1: Introduction.....	4
1.1 Background.....	4
1.2 Motivation.....	4
1.3 Problem Statement.....	4
1.4 Project Objectives.....	4
1.5 Scope of the Project.....	5
1.6 Significance of the Project.....	5
Chapter 2: Literature Review.....	6
2.1 Overview.....	6
2.2 Research Gap.....	7
2.3 Conclusion.....	7
Chapter 3: System Analysis and Requirements.....	8
3.1 Requirement Analysis.....	8
3.2 Functional Requirements.....	8
3.3 Non-Functional Requirements.....	9
3.4 System Feasibility.....	10
Chapter 4: System Design and Architecture.....	11
4.1 System Context.....	11
4.2 Component Architecture.....	11
4.3 Data and Storage Architecture.....	12
4.4 Self-Healing Loop Pipeline.....	13
4.5 Deployment and Infrastructure.....	13
4.6 Use Case Diagrams.....	14
4.7 Data Flow Diagrams.....	15
4.8 Self-Healing Loop Sequence Diagram.....	17
Chapter 5: Implementation.....	19
5.1 Tools and Technology.....	19
5.2 Project View.....	19
5.3 Database Schema.....	28
Chapter 6: Testing and Results.....	30
6.1 Static Verification.....	30
6.2 Live End-to-End Recovery Runs.....	30
6.3 Live Deploy-Engine Verification.....	30
6.4 Defects Found and Fixed During This Engineering Cycle.....	31
6.5 Testing Summary.....	31
Chapter 7: Conclusion and Future Work.....	32
7.1 Overview.....	32
7.2 Limitations of the Project.....	32
7.3 Future Work.....	33
7.4 Conclusion.....	33
References.....	34

Chapter 1: Introduction

This chapter presents the background, motivation, problem, objectives, scope, and significance of **Nexis**.

1.1 Background

Software rarely fails when written; it fails in production, under load, against unanticipated data and dependencies. Industry experience, cited in the project proposal, places 40–60% of engineering time on maintenance, debugging, and incident response rather than new capability — roughly one engineer repairing for every engineer building.

Assistants such as GitHub Copilot [1] now write a meaningful fraction of new code, yet each accelerates a single task for a single human. The production incident lifecycle — detect a fault, diagnose the root cause, fix, validate, obtain approval, deploy, verify — remains a chain of manual hand-offs with the engineer as transport between every stage.

Nexis closes that chain, modelling an engineering organisation as nine role-specialised autonomous agents in two layers — a five-agent Execution Team and a four-agent Self-Healing Loop — orchestrated by the Temporal workflow engine [2], with human approval reduced to a single severity-routed decision.

1.2 Motivation

The motivating pain points:

- **No tool closes the loop.** Sentry [14], Datadog [16], and PagerDuty [15] detect failures and page a human; Copilot [1] writes code once asked; nothing connects detection to autonomous repair.
- **The engineer is the bottleneck for every incident** — reading logs, localising, patching, testing, reviewing, deploying — even for near-identically recurring fault classes.
- **Root-cause analysis is unaided.** No mainstream tooling walks the code dependency graph or ranks candidate causes with stated evidence.
- **Repair without validation is dangerous, so it is not attempted.** With no sandboxed path for a machine-generated patch to prove itself, organisations reasonably refuse, and the loop stays open.
- **Approval is all-or-nothing** — gates are absent (risky automation) or heavyweight (full review for a one-line fix); nothing routes human involvement by measured severity.

1.3 Problem Statement

The limitations of existing approaches are structural — isolated tools, no closed loop, a human bottleneck on every decision, no causal root-cause tooling (Neo4j [3] graph traversal, DoWhy [4] causal reasoning), and no safe autonomous deploy path with sandboxed validation and post-deploy rollback.

The problem is therefore: *how can the full production-fault lifecycle be closed autonomously by cooperating role-specialised agents, while keeping a human decision in the loop exactly where, and only where, the severity of the change warrants it?*

1.4 Project Objectives

Nexis set out to build, and this report documents as built:

1. **A nine-agent, two-layer architecture.** A five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) producing plans, minimal-diff patches, property-based tests, manifests, and migrations; and a four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detecting faults, walking the Neo4j [3] dependency graph toward root causes, and shadow-executing every patch in a locked-down Docker [10] sandbox — orchestrated as a durable, replayable Temporal workflow [2].

2. **Severity-routed human approval.** An Approval Gate classifies each validated patch HIGH (mandatory human review), MEDIUM (120-second intervention window, then auto-approval), or LOW (auto-approval); Approve, Reject, and diff-editing Modify decisions are persisted as feedback.
3. **Real GitOps closure.** Approval opens a genuine GitHub pull request via a GitHub App; a post-deploy probe watches for fresh fatal errors, invoking Argo CD rollback [6] to the prior known-good revision when recovery policy demands.
4. **A Docker preview-deploy engine** that detects a cloned repository's stack, generates a missing Dockerfile, builds and runs the container under strict resource limits, health-checks it over HTTP, and returns a live preview URL, feeding its failures back as first-class incidents.
5. **Intelligent role recommendation.** A heuristic engine over audit-log and login signal recommends privilege escalations and downgrades, each with an evidence-backed rationale to accept or dismiss.
6. **A multi-tenant platform around the loop:** PostgreSQL Row-Level Security (RLS) [7], Role-Based Access Control (RBAC), MFA and SSO via WorkOS [12], Stripe billing [13], integrations (GitHub, Slack [17], Sentry [14], Datadog [16], PagerDuty [15]), a tamper-evident audit log, and an administrative console.

1.5 Scope of the Project

Nexis serves three human roles under RBAC:

- **Owner/Administrator** — provisions the organisation, connects integrations, sets recovery policy, decides HIGH-severity approvals, manages members and billing, acts on role recommendations.
- **Engineer/Member** — works daily with projects and incidents, observes live pipeline runs, exercises Approve/Reject/Modify.
- **Viewer** — read-only console visibility, RBAC-enforced, into incidents, recovery outcomes, and cost.

Beyond these, the **nine agents are modelled as a first-class actor category**: the Sentinel raises incidents and the Validator renders verdicts as human actors would, under the same audit and policy constraints — shrinking the human roles to supervisory decisions.

The delivered scope is the platform above, evaluated end-to-end against a locally hosted Large Language Model via Ollama [20]; the `nexis.sh` domain is registered, but a credentialed public production deployment is outside this report's scope, as the limitations discuss honestly.

1.6 Significance of the Project

Nexis's significance is the closed loop itself: one continuous, durable, auditable workflow from detection to verified deployment, the human reduced from transport mechanism to a single well-placed decision.

Chapter 6 evidences this live: two end-to-end recovery runs — one correctly rejecting the patch on approval-window timeout, the safety mechanism working as designed; one carrying a fault from detection to an approved, deployed patch in under two minutes, with per-agent token and cost figures — and an HTTP-verified preview-deploy of a Dockerfile-less public repository.

Chapter 2: Literature Review

2.1 Overview

Nexis is a systems-integration project: it composes established tools into a closed loop none forms alone; the survey marks, per tool, dependency versus extension.

Large Language Model (LLM) coding assistants. GitHub Copilot [1] completes code and drafts multi-file changes in-editor, yet never observes a production incident, never validates against a failing system, and never deploys. Nexis’s Backend agent fills the same niche, invoked by a fault-detection pipeline rather than a keystroke, machine-validated before any human sees it.

Durable workflow orchestration. Temporal [2] supplies durable, replayable workflow execution with retries and state persistence but no engineering semantics. The nine-agent RecoveryPipeline is a Temporal workflow, so agent steps survive restarts as durable ActivityEvents; the role routing, contract-violation detection, and severity classification are Nexis’s logic.

Causal inference and graph analysis. Neo4j [3] stores the code-dependency graph the Pathfinder agent walks from symptom to candidate root causes; the evidence ranking is Nexis code. DoWhy [4], with its refutation tests, is the sidecar’s methodological reference, but Nexis does *not* fit a DoWhy structural causal model: no interventional production data exists. The sidecar instead ranks candidates by a documented proxy (evidence-chain specificity, graph proximity, textual overlap, scenario prior) with a transparent per-component confidence breakdown.

The self-healing/AIOps ecosystem. Sentry [14], Datadog [16], and PagerDuty [15] capture errors, aggregate telemetry, and page on-call humans; none generates a fix, let alone validates one. All three feed Sentinel as webhook incident sources; the Slack API [17] carries the reverse, one-click Approval-Gate approve/reject of validated patches.

Further dependencies. Argo CD [6] handles GitOps deploys, its real Rollback client firing on a policy-gated SLO breach. Docker [10] backs the Validator’s network-isolated sandbox and preview deploys; Podman [11], the API-compatible evaluation substitute, behaved identically, reported honestly. Hypothesis [9] sees novel use: the Quality Assurance (QA) agent generates pytest/Hypothesis suites from incident and diff; a background loop replays them, raising incidents on pass-to-fail regressions. The Data Engineer agent emits OpenLineage [8] RunEvents per migration. OpenTelemetry [5], WorkOS [12], Stripe [13], Row-Level Security [7], pgvector [19], and Ollama [20] (the report’s live-evidence LLM) are used as documented.

Table 2.1: Surveyed tools relative to Nexis.

Tool	Relation to Nexis
GitHub Copilot [1]	Backend-agent analogue only
Temporal [2]	Hosts RecoveryPipeline
Neo4j [3]	Dependency; Pathfinder traversal
DoWhy [4]	Reference; proxy used
OpenTelemetry [5]	Instrumentation dependency
Argo CD [6]	Deploy, rollback probe
Docker/Podman [10, 11]	Sandbox, deploy engine
Hypothesis [9]	Novel use: QA suites
OpenLineage [8]	Data Engineer conformance
Sentry/Datadog/PagerDuty [14, 16, 15]	Webhook sources
Slack API [17]	One-click approvals

2.2 Research Gap

Industry routinely chains these tools, yet none, alone or combined, both (a) models a software engineering *team* with explicit role specialisation and (b) closes the loop from automatic fault detection to a deployed, verified fix with human judgement concentrated at one severity-routed decision point. The break is always the same: between “a fault is known” and “a validated fix exists,” a human performs diagnosis, authoring, testing, review, and release unaided.

Nexis closes that span: Sentinel detects faults via an Exponentially Weighted Moving Average (EWMA) baseline breached at mean-plus-three-sigma. The Synthesiser routes only needed agents, the workflow visibly skipping the rest. The Architect’s `affected_files` contract is mechanically checked against the Backend diff; a violation forces HIGH severity, so no change auto-deploys unreviewed. QA generates property-based tests [9]; the Validator executes patch and tests sandboxed, so no unvalidated patch reaches a human. One decision point engages: HIGH requires review, MEDIUM auto-approves after 120 seconds unless overridden, LOW passes immediately. Engineers may Approve, Reject, or Modify, decisions persisted for preference learning. Approval opens a real GitHub pull request under a post-deploy rollback probe [6].

Two properties distinguish it from scripted tool-chaining. Role specialisation is structural: five Execution Team agents and four Self-Healing Loop agents have distinct inputs, outputs, and contracts under the Architect’s plan. And the loop closes over Nexis’s own infrastructure: a preview deploy that fails its build or health check re-enters the pipeline as a first-class incident, so “write a working Dockerfile” becomes an ordinary repair task. To the author’s knowledge, no tool in common use combines these properties in a single system.

2.3 Conclusion

The landscape supplies every ingredient yet leaves a human to close the detection-to-repair loop. Nexis’s contribution is not a new algorithm but the composition of a role-specialised multi-agent team that detects, diagnoses, repairs, validates, and deploys autonomously, engaging human judgement once at a severity-routed gate. Later chapters describe its specification, design, implementation, and testing.

Chapter 3: System Analysis and Requirements

3.1 Requirement Analysis

Nexis is a role-based, multi-tenant platform: every datum belongs to exactly one organisation, and every request is evaluated against the caller's organisation and role; analysis begins with the actors at the system boundary. Three human actor types were identified:

- **Owner/Administrator** — owns membership, roles, billing, policy, integrations, and the final human decision on high-stakes automated repairs.
- **Engineer/Member** — investigates incidents, exercises the evaluation tooling, works the recovery pipeline daily.
- **Viewer** — a read-only observer (stakeholder or auditor): inspects system state, never changes it.

The role model is *additive* — each tier holds every permission of the tier below — so Section 3.2 lists only what each role *adds*. (Internally: `owner/admin/member`; Viewer is RBAC-gated on frontend and API.)

A fourth actor is not human: the **nine-agent self-healing fleet**. It acts autonomously inside the boundary — detecting faults unprompted, writing the console's own PostgreSQL, Neo4j, and object-storage stores, and triggering real side effects: GitHub pull requests, Argo CD rollbacks [6], preview redeploy. Roughly half of all incident, patch, and audit-table writes are agent-authored; omitting the fleet would misrepresent the system, and modelling it makes the safety analysis explicit: every Section 3.2.4 capability is bounded by a Section 3.3 control — sandboxing, severity classification, or the human approval gate.

The four actors' Level 0/1 use case and data flow diagrams appear in Sections 4.6–4.7.

3.2 Functional Requirements

3.2.1 Owner/Administrator can:

- Manage membership and roles, including the role-recommendation banner: accept applies under an optimistic-concurrency guard, dismiss starts a 30-day cool-down.
- Manage Stripe billing [13] and daily token budgets; export/verify the hash-chained audit log; get the daily digest; monitor System Health.
- Decide approvals — **Approve**, **Reject**, or **Modify** (diff edits persist as RLHF feedback) — and set recovery policy, including auto-rollback on SLO breach.
- Operate the preview-deploy engine; install GitHub App, Sentry [14], Datadog [16], PagerDuty [15], Slack [17] (in-message approve/reject), and Argo CD [6]; export evaluation CSVs.

3.2.2 Engineer/Member can:

- Authenticate via email/password (HttpOnly JWT cookie), magic link, or WorkOS SSO [12]; hashed, time-limited reset tokens; TOTP MFA; remote session revocation; scoped `nx_live_...` API keys.
- Create GitHub-connected projects; follow the nine-agent timeline over Server-Sent Events (SSE), inspecting per-agent status, tokens, cost, structured payloads.
- Trigger fault injection from 27 Live-Demo scenario cards (one wired end-to-end today, the rest “Coming soon”).
- Run the OpenAI-versus-Ollama evaluation harness (persisted transcripts, costs); query the pgvector knowledge base [19]; complete the post-incident NASA-TLX survey [18].

3.2.3 Viewer can:

- Sign in and read: dashboard KPIs (MTTR, success rate, open incidents, token spend), live incident timelines, and project, performance, agents, and activity pages — never mutating anything.

3.2.4 The Nine-Agent Fleet autonomously:

- **Detects** (Sentinel): a per-row fatal trigger plus an EWMA spike rule (mean + 3σ , Poisson floor, warm-up fallback).
- **Diagnoses** (Pathfinder): walks the Neo4j graph [3]; the causal sidecar ranks candidates by evidence specificity, proximity, overlap, and scenario prior [4].
- **Routes** (Synthesiser): assigns a scenario (regex fast path, LLM fallback) and an enforced `selected_agents` list.
- **Plans** (Architect): `plan_steps`, `affected_files`, `risk_level` form a contract — diffs beyond `affected_files` force HIGH severity, never auto-deploying.
- **Codes** (Backend) a minimal LLM-guided unified diff; **Tests** (QA) with auto-generated pytest/Hypothesis property suites [9], replayed continuously, regressions raising fresh incidents.
- **Packages** (DevOps): CI/CD and Argo CD manifests [6]; **handles data** (Data Engineer): paired forward/reverse SQL migrations, `information_schema` drift detection, OpenLineage events [8].
- **Validates** (Validator): every patch runs in a Docker sandbox [10] (no network, read-only filesystem, dropped capabilities, PID limit, `no-new-privileges`) before the approval gate.
- **Classifies severity**: HIGH (schema drift, sensitive paths, unknown scenarios, contract violations); MEDIUM (120-second auto-approve countdown); LOW (tiny UI-only diffs) auto-approves.
- **Deploys** (GitOps) via real GitHub pull requests; **rolls back** via Argo CD on post-deploy regression, policy permitting; **records** each step as a Temporal-durable SSE-streamed event [2]; and refiles deploy-engine build failures as incidents: broken builds self-heal too.

3.3 Non-Functional Requirements

Table 3.1: Non-functional requirements and realisation.

Requirement	Realisation in Nexis
Multi-tenancy	PostgreSQL Row-Level Security (RLS) on essentially every table [7]
Durability	Temporal-orchestrated workflows survive crashes and restarts [2]
Security	Sandboxed patch execution, encrypted patch storage, hashed tokens, hash-chained audit log
Scalability	Stateless Go services, queue-decoupled workers, parallel dependency probes
Responsiveness	Server-Sent Events stream the live incident timeline
Usability	Role-scoped console, one-click approvals, plain-language rationales

Isolation must not depend on code remembering `WHERE org_id`: RLS policies [7], bound per request to `app.current_org_id`, make the database itself refuse cross-tenant access, humans and agents alike. A recovery run spans minutes and awaits a human decision, so pipeline state lives in Temporal [2] as journaled activities replayed deterministically after a crash — an in-memory orchestrator would strand incidents mid-repair, and was rejected. Generated code is untrusted by construction: patches execute only sandboxed, patch blobs are encrypted at rest, tokens exist only as SHA-256 hashes (30-minute TTL, non-leaky 202), and audit-log tampering detectably breaks the chain.

3.4 System Feasibility

3.4.1 Technical Feasibility

Nexis is deliberately polyglot. Four independent backend Go services get static binaries, cheap concurrency, and a first-class Docker API client for webhook and container work. Next.js/React/TypeScript serves the role-gated, SSE-streaming, Monaco-editing console; the Python/FastAPI causal sidecar sits on the scientific-Python/Hypothesis [9] stack. PostgreSQL 17 with pgvector [19] supplies transactions, RLS, and vector retrieval in one engine; Neo4j [3] holds the dependency graph because diagnosis is graph traversal, poorly expressed relationally. Temporal [2] was proven by both live recovery runs in Chapter 6; everything was built and verified on one development machine, Podman [11] substituting for Docker [10].

3.4.2 Economic Feasibility

Every dependency is open source (PostgreSQL, Neo4j Community, Redis, MinIO, Temporal, OpenTelemetry [5], Grafana) — no licence cost. LLM inference, the one significant cost, is dual-path: OpenAI models for production, local Ollama [20] models (`gpt-oss:20b`, `qwen2.5-coder:14b`) for development. Every live run in this report used the local provider — the complete system exercised at zero token cost; the cost tracker and daily budgets bound the paid path.

3.4.3 Operational Feasibility

Administrators land on organisation-wide KPIs, approvals, and health; engineers on incidents and the live pipeline; viewers on the same truth, read-only. High-stakes autonomy reduces to one-click Approve/Reject/Modify, optionally from Slack [17]; severity classification lets low-risk fixes pass unattended while sensitive changes never do. The built-in NASA-TLX instrument [18] lets the operator-workload claim eventually be measured, not asserted; the intended operator is an ordinary engineer with a browser — no dedicated operations team.

Chapter 4: System Design and Architecture

This chapter presents the design of Nexis at complementary levels: the structural view (Sections 4.1–4.5), the functional view via use case diagrams (4.6), the data view via Data Flow Diagrams (4.7), and a UML sequence diagram of one complete recovery run (4.8). The self-healing loop is given the most space, because it is the central contribution of the project.

4.1 System Context

Figure 4.1 places Nexis at the centre of its environment: the three human roles (Owner/Administrator, Engineer/Member, Viewer), the nine-agent fleet as a fourth, autonomous actor, and the external systems — GitHub, Slack [17], the LLM provider (OpenAI or Ollama), Argo CD, Stripe [13], alert sources Sentry [14], Datadog [16], and PagerDuty [15], WorkOS SSO [12], and SES/SMTP mail.

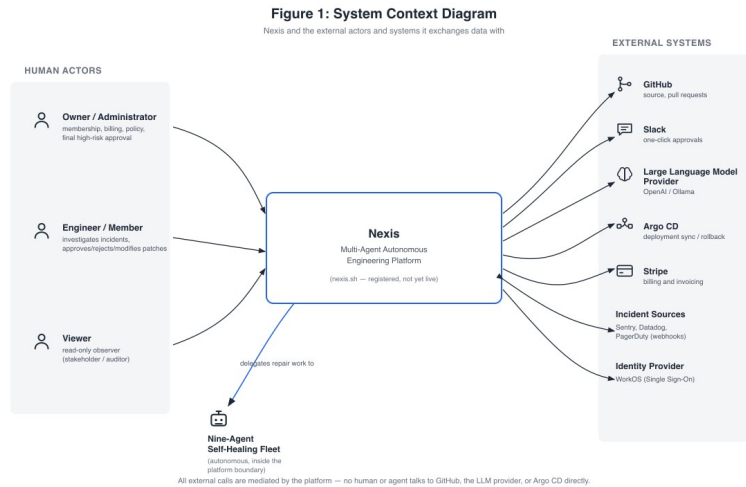


Figure 4.1: System Context Diagram. (Nexis and the external actors and systems it exchanges data with).

Every entry point (web console, webhooks, Slack callbacks, plain HTTP clients) converges on the control-plane — no human or agent talks to a data store or external integration directly. An honest status note: the `nexis.sh` domain is registered, but the platform is not yet publicly live (Section 4.5).

4.2 Component Architecture

Figure 4.2 opens the platform boundary: the Next.js 16 web console, and at the centre a single Go control-plane (:8080) hosting the chi-routed API layer (authentication, RBAC, RLS binding, rate limiting), the always-on Sentinel detector (EWMA +3 σ spike rule), and the durable Temporal RecoveryPipeline workflow [2] orchestrating the nine-agent loop.

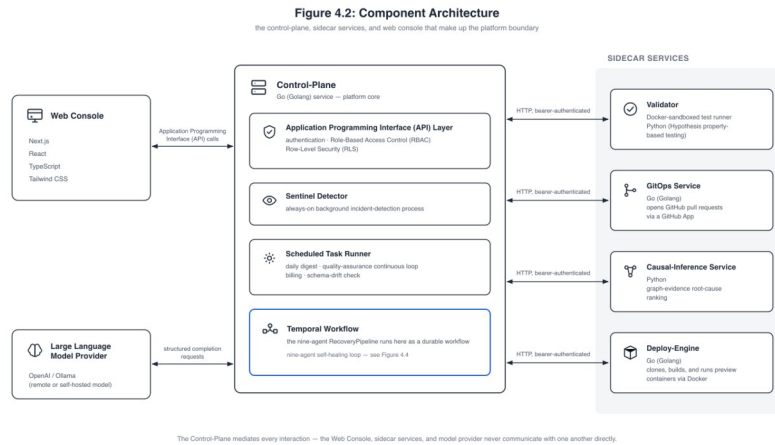


Figure 4.2: Component Architecture. (the control-plane, sidecar services, and web console that make up the platform boundary).

Around it sit four bearer-authenticated sidecars: the Docker-sandboxed Validator (:8081) [10], the GitOps PR service (:8082), the Python/FastAPI causal-inference ranker (:8090 — an honest statistical proxy, not a fitted structural causal model [4]), and the deploy-engine (:8091), plus an Argo CD Sync/Rollback client [6]. The LLM spine pairs OpenAI (gpt-4o family) with Ollama [20] (gpt-oss:20b, qwen2.5-coder:14b) as a fully supported alternate, backed by pgvector retrieval [19] and per-agent JSON Schema validation with retry; the control-plane mediates every interaction.

4.3 Data and Storage Architecture

Figure 4.3 shows the four storage systems plus Temporal’s workflow persistence layer.

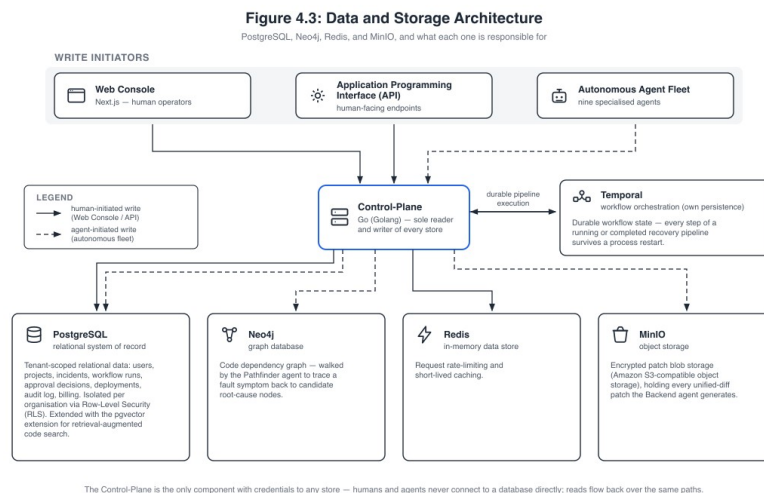


Figure 4.3: Data and Storage Architecture. (PostgreSQL, Neo4j, Redis, and MinIO, and what each one is responsible for).

PostgreSQL 17 with pgvector is the system of record, multi-tenant via Row-Level Security [7] across 33 migrations; Neo4j [3] holds each repository’s dependency graph for Pathfinder; Redis serves rate limiting and caching; MinIO is the encrypted patch store. The footer’s isolation claim is the key point: only the control-plane holds store credentials — humans and agents never connect to a database directly.

4.4 Self-Healing Loop Pipeline

Figure 4.4 presents the central contribution as a single flow — the nine agents in order: Sentinel (detect), Pathfinder (localise via the Neo4j graph), Synthesiser (classify, select agents), Architect (plan and file-level contract), Backend (minimal patch), Quality Assurance (property-based tests), DevOps in parallel with Data Engineer (manifests and migrations), and Validator (sandboxed shadow-execution).

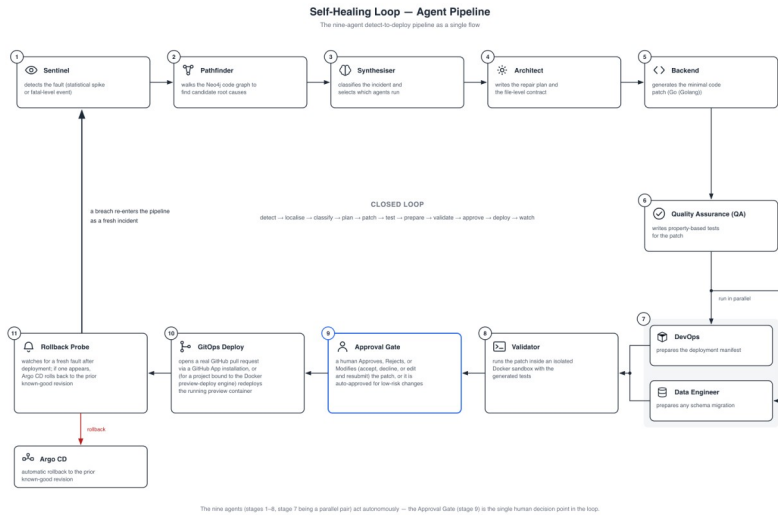


Figure 4.4: Self-Healing Loop — Agent Pipeline. (the nine-agent detect-to-deploy pipeline as a single flow).

Only then does a person enter: the Approval Gate (approve, reject, or modify; auto-approved for low-risk changes) is the single human decision point. GitOps Deploy then opens a real pull request or redeploys a preview container, and the Rollback Probe rolls back via Argo CD on a fresh post-deploy fault, feeding the breach back into detection.

4.5 Deployment and Infrastructure

Figure 4.5 shows how the platform is built, delivered, and observed: GitHub Actions CI, Docker (Podman-compatible) as the universal runtime, Argo CD GitOps delivery, and Terraform declaring the AWS infrastructure as code.

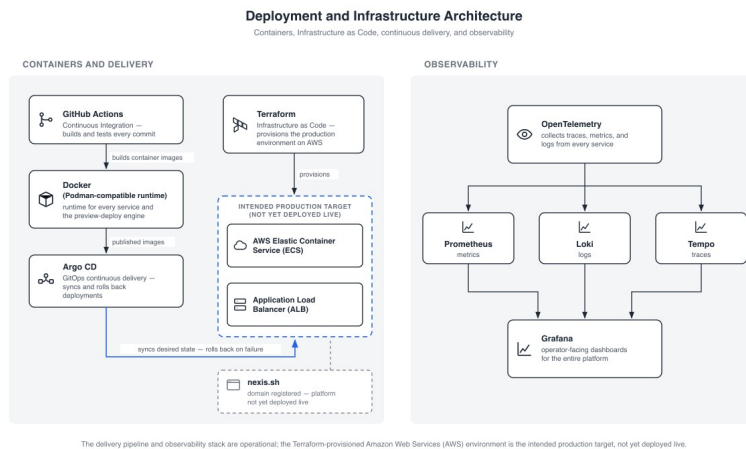


Figure 4.5: Deployment and Infrastructure Architecture. (containers, Infrastructure as Code, continuous delivery, and observability).

The diagram is deliberately honest about status: the AWS ECS cluster behind an Application Load Balancer is the *intended* production target declared in Terraform — not yet deployed live — and the registered `nexis.sh` domain

does not yet serve the platform. Observability follows OpenTelemetry [5] into Prometheus, Loki, and Tempo, visualised in Grafana; the evaluation environment ran the sandbox and deploy-engine on Podman [11], exercising its Docker-API compatibility in practice.

4.6 Use Case Diagrams

4.6.1 Use Case Diagram, Level 0

Figure 4.6 shows the coarsest functional view: the three human actors and the autonomous nine-agent fleet against a single system boundary, with the mediated external systems on the right.

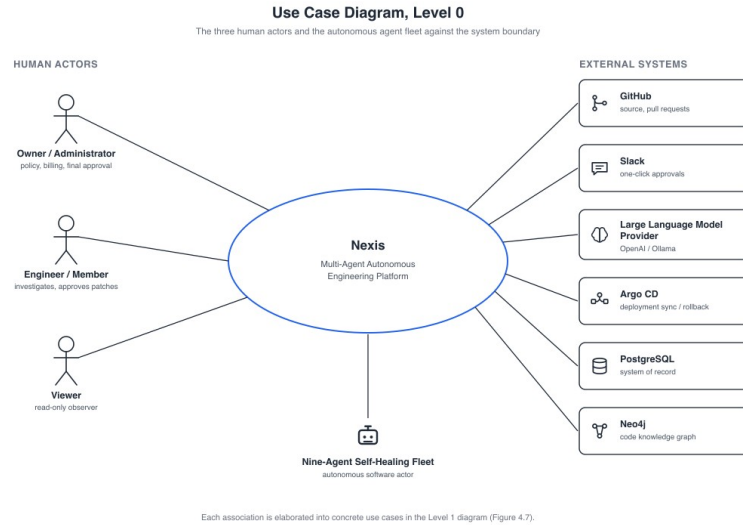


Figure 4.6: Use Case Diagram, Level 0. (the three human actors and the autonomous agent fleet against the system boundary).

The essential property is mediation: every association terminates at the system boundary — no human ever calls GitHub, the LLM provider, or Argo CD directly; the agents act on the external systems on the humans’ behalf, surfacing back to a person only at the approval gate.

4.6.2 Use Case Diagram, Level 1

Figure 4.7 expands the Level 0 bubble into concrete use cases grouped by minimum role, with generalisation arrows making role inheritance explicit.

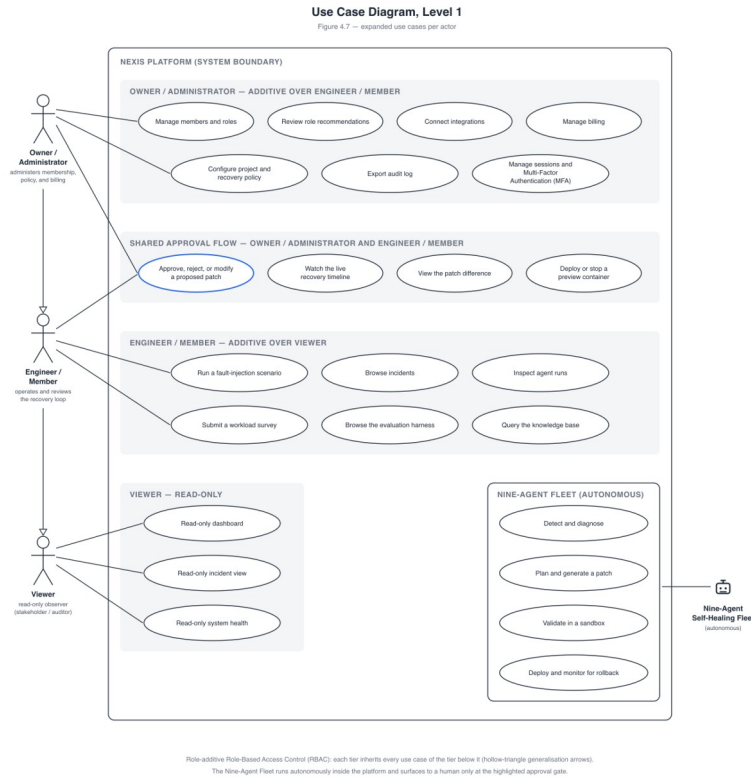


Figure 4.7: Use Case Diagram, Level 1. (expanded use cases per actor).

The administrative tier holds fifteen use cases (membership, role recommendations, integrations, recovery policy and kill switch, billing, the RLHF/JSONL feedback export, audit, MFA); four approval-flow use cases are shared with the Engineer/Member, whose tier includes demo scenarios, agent runs, and the NASA-TLX survey [18]; the Viewer holds exactly three read-only use cases. The model is role-additive — each tier inherits every use case of the tier below — and the read-only Viewer experience is enforced by RBAC gating in the console even though the database role enumeration is owner/admin/member.

4.7 Data Flow Diagrams

4.7.1 Data Flow Diagram, Level 0 (Context)

Figure 4.8 is the context diagram: the entire platform as a single process 0 with three top-level stores — D1 the tenant database (PostgreSQL, under Row-Level Security), D2 the code graph (Neo4j), and D3 the encrypted patch store (MinIO).

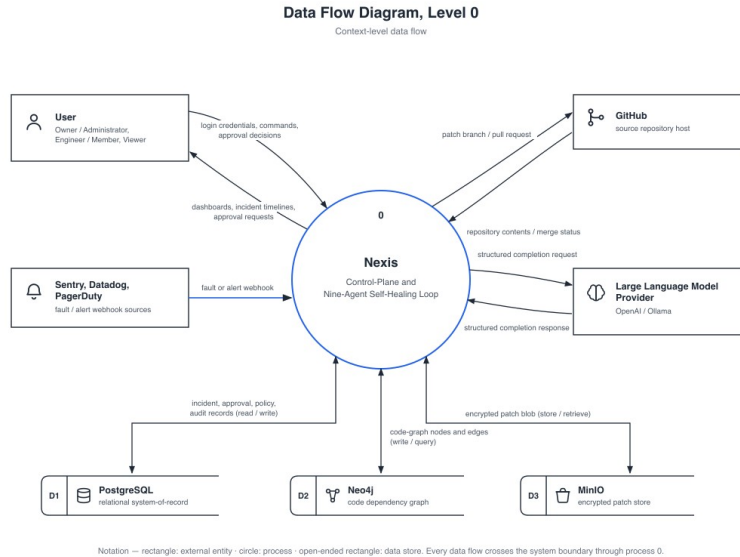


Figure 4.8: Data Flow Diagram, Level 0. (context-level data flow).

Four entity flows cross the boundary — the user, GitHub, the alert sources (producers of incidents, not consumers), and the LLM provider — and every one is mediated by process 0: there is no direct path from a human to an integration.

4.7.2 Data Flow Diagram, Level 1 — Owner/Administrator

Figure 4.9 decomposes the administrative surface into ten numbered processes — spanning identity and access, platform administration, and incident and approval operations — over the tenant-scoped PostgreSQL stores; Table 4.1 summarises the most essential.

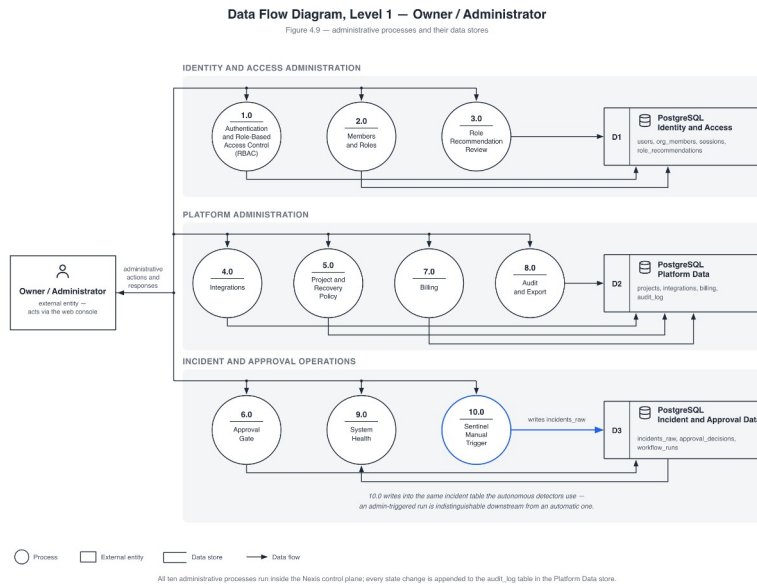


Figure 4.9: Data Flow Diagram, Level 1 — Owner / Administrator. (administrative processes and their data stores).

Table 4.1: Owner/Administrator Data Flow Diagram Level 1 processes and their primary data stores.

Process	Responsibility	Primary store written
1.0 Authentication and RBAC	login, JWT session issue, role checks	users, sessions
5.0 Project and Recovery Policy	recovery policy, SLO, kill switch	projects
6.0 Approval Gate	approve / reject / modify decisions	approval_decisions
8.0 Audit and Export	hash-chained audit writes, CSV export	audit_log
10.0 Sentinel Manual Trigger	administrator-initiated detection run	incidents_raw

The most important flow is 10.0: a manual detection run writes into the *same* `incidents_raw` table as the automatic detectors — no separate code path — so every autonomous-path guarantee (durability, validation, severity routing, auditability) holds for manual runs for free. 5.0 carries the kill switch that suspends autonomous recovery, and every state change lands in the hash-chained `audit_log`.

4.7.3 Data Flow Diagram, Level 1 — Self-Healing Loop

Figure 4.10 decomposes the loop into twelve numbered processes — detection and decision, the Layer-1 execution team, and the validation–approval–deployment–verification tail — over five stores (D1 incidents, D2 code graph, D3 patch store, D4 approvals and feedback, D5 deployments and lineage).

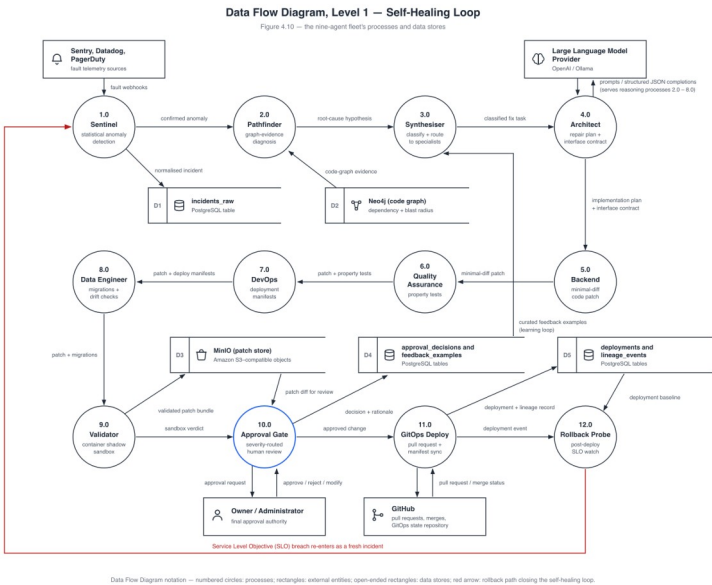


Figure 4.10: Data Flow Diagram, Level 1 — Self-Healing Loop. (the nine-agent fleet’s processes and data stores).

The closing annotation states the property that gives the project its title: the loop is closed. An SLO breach seen by the rollback probe (12.0) lands in `incidents_raw` as a fresh fatal and re-enters at Sentinel (1.0) — with an Argo CD rollback to the prior known-good revision when `RollbackOnSLOBreach` is set — so recovery, verification, and re-detection form one cycle over one shared data path.

4.8 Self-Healing Loop Sequence Diagram

Figure 4.11 traces one complete recovery run across all nine agent lifelines in eighteen numbered messages — each a named Temporal activity, durably recorded and streamed to the incident timeline over Server-Sent Events, the exact order the Chapter 5 screenshots show.

Chapter 5: Implementation

This chapter documents the implemented system as it exists and runs today: the technology stack, a screen-by-screen walk through the platform, and the database schema beneath the multi-tenant control plane. All screenshots are unretouched captures of the running development environment; errors and pending states are reported as-is rather than substituted with more flattering captures.

5.1 Tools and Technology

Nexis is a polyglot pnpm + Turborepo monorepo, each subsystem in the language best suited to its role:

- **Backend** — Go 1.25 in four modules: `services/control-plane` (multi-tenant API), `services/validator` (sandboxed patch validation), `services/gitops` (GitHub-App deployment), `services/deploy-engine` (Docker preview deploys); a Python/FastAPI sidecar, `services/causal-inference` (port 8090), ranks candidate root-cause nodes for the Pathfinder agent; Temporal [2] durably orchestrates the nine-agent RecoveryPipeline — every step an ActivityEvent, crashed workers resume in place.
- **Frontend** — Next.js 16 / React 19 / TypeScript (`apps/web`); Tailwind CSS 4 and Radix UI; Monaco for unified-diff editing; Recharts; Server-Sent Events for live incident pages without polling.
- **Data** — PostgreSQL 17 (system of record, Row-Level Security (RLS) [7], 33 migrations); pgvector [19] (knowledge-base retrieval); Neo4j [3] (code dependency graph); Redis (rate limiting, caching); MinIO (S3-compatible encrypted patch blobs); Temporal's own durable event history.
- **AI / LLM** — OpenAI (GPT-4o family) is the production-configured provider; Ollama [20] (`gpt-oss:20b`, `qwen2.5-coder:14b`) is fully supported — all live-run evidence in this chapter used Ollama end-to-end, with no cloud Large Language Model (LLM) credentials. The Quality Assurance (QA) agent generates Hypothesis [9] property-based tests that the Validator executes in its sandbox. The causal-inference sidecar is a statistical/graph ranking algorithm, not a fitted DoWhy [4] model — that would need interventional production data that does not yet exist, a limitation stated openly.
- **DevOps** — Docker API [10] for both the Validator sandbox and preview deploys (Podman-compatible [11]; Podman substituted in the evaluation environment worked identically); Terraform to AWS ECS behind a load balancer; OpenTelemetry [5] to Prometheus, Loki, Tempo, and Grafana; Argo CD [6] for the DevOps agent's Sync/Rollback client.
- **Version control** — Git and GitHub; a GitHub App installation supplies short-lived tokens for cloning and the GitOps blob → tree → commit → ref → Pull Request deployment path.

5.2 Project View

The captures below follow the order a new organisation would encounter the platform.

5.2.1 Authentication and Onboarding

Access is a local JSON Web Token (JWT) session (HttpOnly cookie) with optional WorkOS [12] Single Sign-On. Sign-up (Figure 5.1) creates the account; forgot-password issues a 32-byte reset token, SHA-256-hashed with a 30-minute time-to-live, and returns the same 202 response whether or not the e-mail exists, leaking no account information.

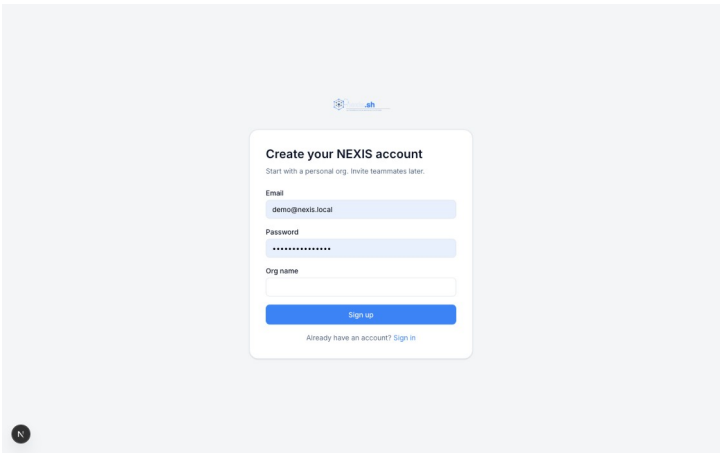


Figure 5.1: Sign-Up Screen. (new account registration).

5.2.2 Administrator / Owner Console

The console home (Figure 5.2) shows KPI tiles for Mean Time To Recovery (MTTR), recovery success rate, open incidents, and mean tokens per run, with an incidents-over-time chart, system status, projects health grid, and activity feed.

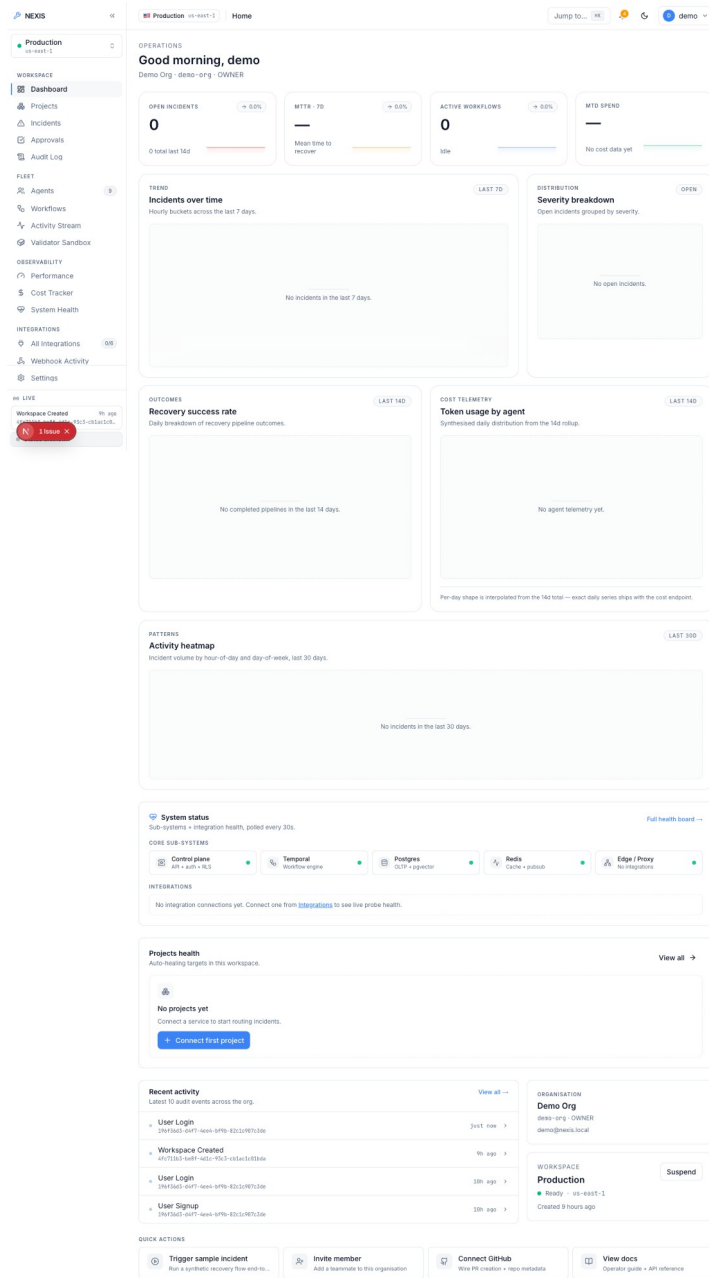


Figure 5.2: Administrator Dashboard. (KPI tiles, incidents-over-time chart, and system status panel).

The Agents page (Figure 5.3) lists the nine-agent fleet across both layers — the five Execution Team agents (Architect, Backend, QA, DevOps, Data Engineer) and the Self-Healing Loop agents (Sentinel, Pathfinder, Synthesiser, Validator); the Incidents page (Figure 5.4) lists every ingested incident.

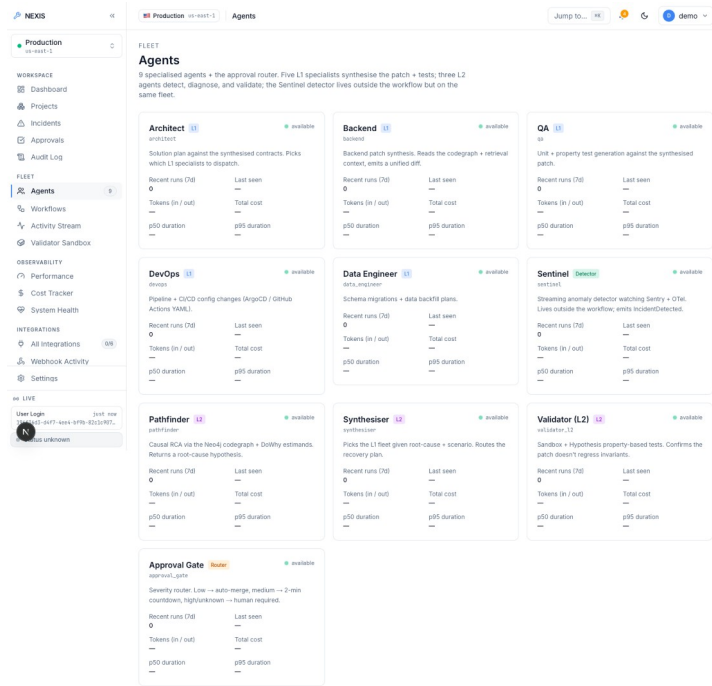


Figure 5.3: Agents Screen. (the nine-agent fleet roster across both layers).

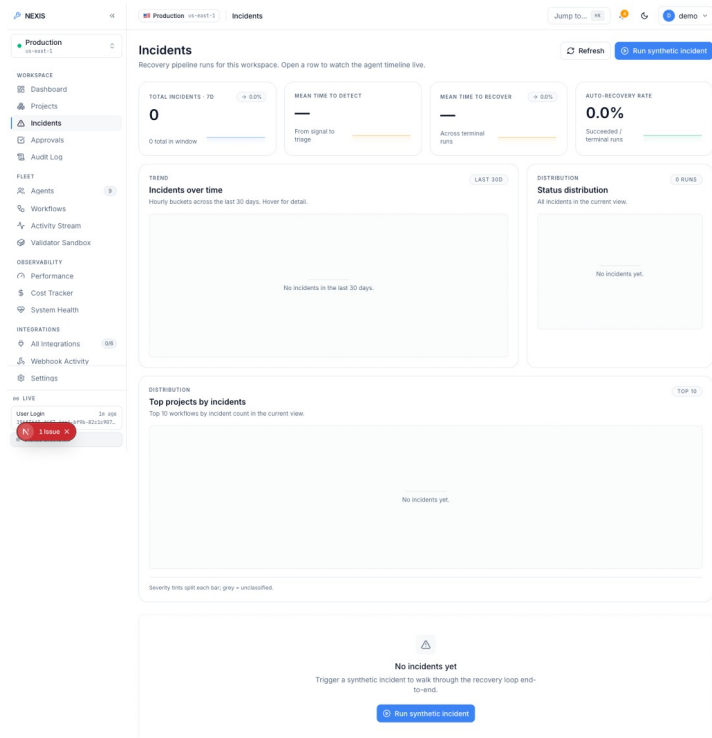


Figure 5.4: Incidents Screen. (the organisation-wide incident list).

Settings cover members with `owner` / `admin` / `member` roles and a real heuristic role-recommendation panel (evidence-backed rationales, 30-day cool-down on dismissal), remotely revocable sessions, scoped API keys (`nx_live_...`), and Stripe billing [13].

5.2.3 Self-Healing Loop — Live Evidence

This subsection is the evidentiary core of the report: two real, end-to-end executions of the nine-agent RecoveryPipeline against a live local LLM (Ollama, qwen2.5-coder:14b and gpt-oss:20b) — one safely auto-rejected at the approval gate, one approved and fully succeeded — shown exactly as they occurred. The Live Demo entry point offers 27 fault-scenario cards; exactly one is fully wired end-to-end today, the rest UI-complete but backend-pending, labelled “Coming soon”. Both runs used the wired path; Figure 5.5 shows the pipeline structure.

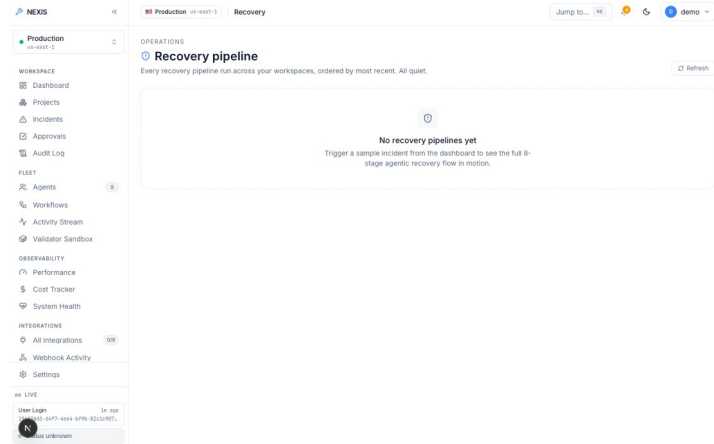


Figure 5.5: Recovery Pipeline Screen. (the nine-agent pipeline structure).

Figure 5.6 captures the first run genuinely mid-flight: Architect, Backend, and QA have completed with real, non-zero token counts and costs from actual LLM calls while DevOps and Data Engineer run in parallel. A stubbed pipeline would report zero tokens; these figures are themselves evidence.

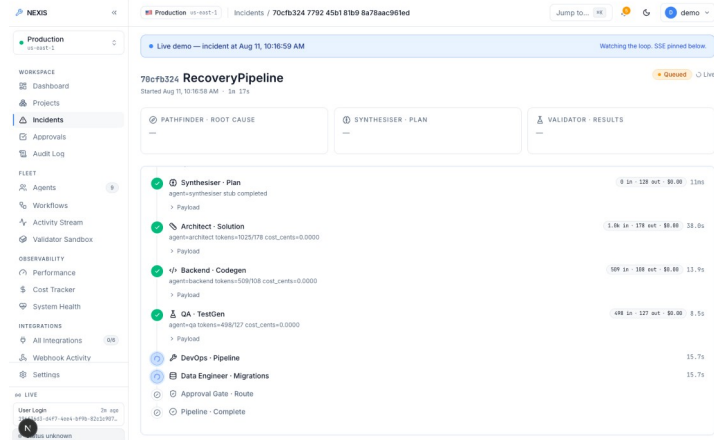


Figure 5.6: Incident Detail, Mid-Flight. (completed Architect/Backend/QA steps with real token counts; DevOps and Data Engineer running in parallel).

Every step then completed green, leaving severity “Medium” and approval “Pending” inside a 120-second window. The window elapsed with no decision and the workflow terminated as `timeout_rejected` — an honest negative result, reported deliberately: absent a human decision, the system refuses to deploy unreviewed code rather than proceeding.

The second run, approved within the window, is the key positive result (Figure 5.7): “Medium” severity, “Approved”, “Succeeded”, the full nine-step timeline completing in 1 minute 40 seconds with real per-agent token and cost figures.

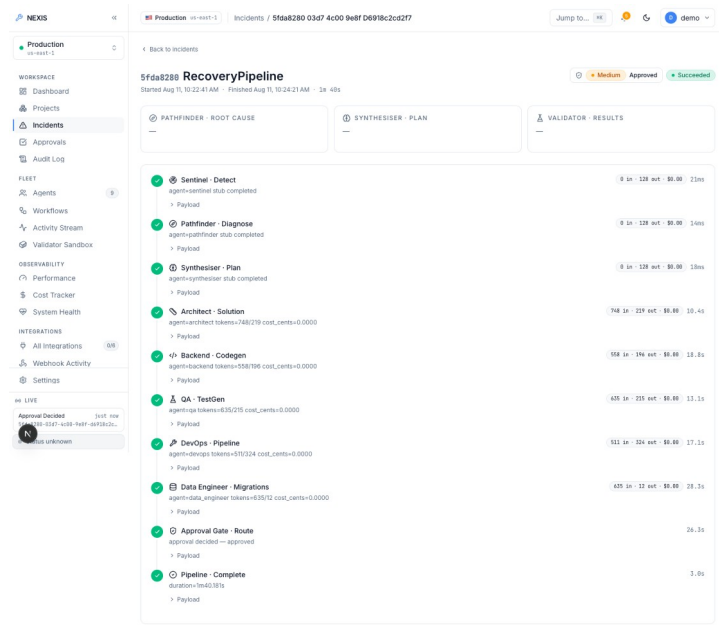


Figure 5.7: Fully Approved Run. (Medium severity, Approved, Succeeded; complete nine-step timeline in 1 minute 40 seconds).

Pending runs surface in the Approvals queue (Figure 5.8) with three human actions — Reject, Modify, Approve — decided in the dialog of Figure 5.9. Modify is the Reinforcement Learning from Human Feedback (RLHF) mechanism made concrete: the engineer edits the LLM-generated unified diff before approving, and the edited diff — with every terminal decision — is persisted to `feedback_examples`, feeding a JSONL export for future fine-tuning; submitting a paid fine-tuning job is a deliberate manual step, not automated.

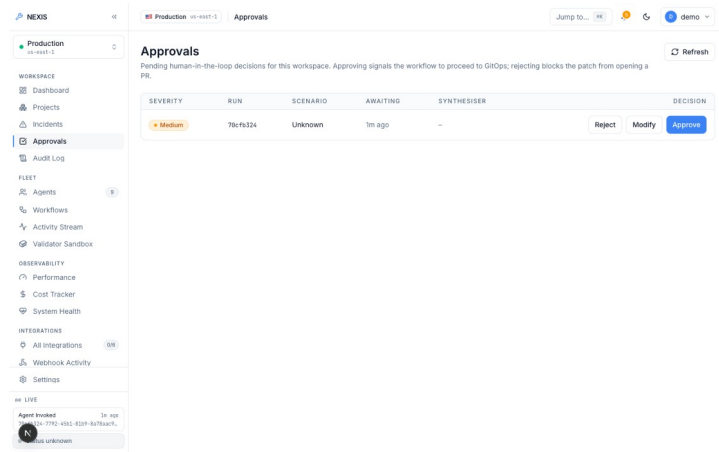


Figure 5.8: Approvals Queue. (real pending approval row with Reject / Modify / Approve actions).

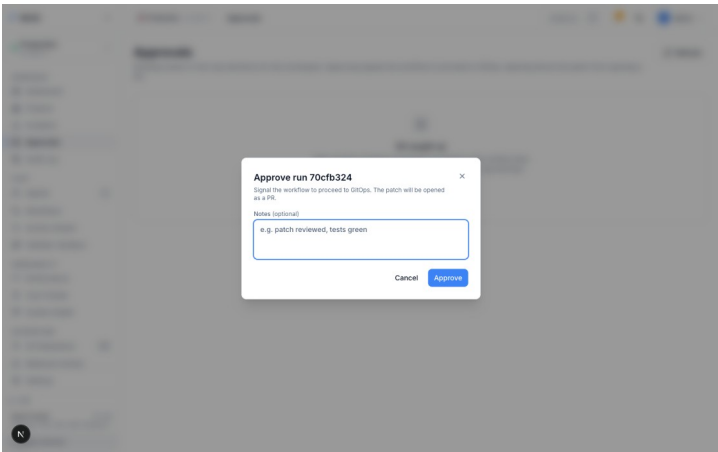


Figure 5.9: Approval Decision Dialog. (the Reject / Modify / Approve decision surface for a pending patch).

The patch is genuinely model-generated: the Backend step’s raw JSON records the unified diff produced by `qwen2.5-coder:14b` with `provider="ollama"` — parsed model output, not a fixture template. Figure 5.10 shows the Backend agent’s own detail page.

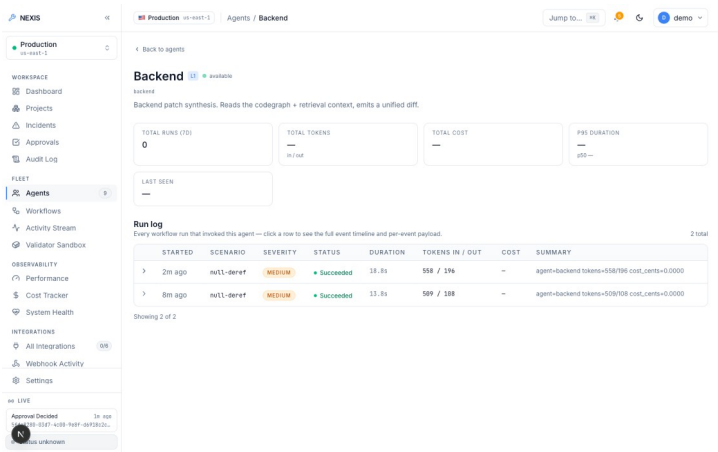


Figure 5.10: Agent Detail, Backend Agent. (per-agent configuration and run history).

5.2.4 Docker Preview-Deploy Engine (Ops Tab)

The project Ops tab fronts the preview-deploy engine (Deploy action, live status, preview URL, logs, history). In Figure 5.11, Deploy on a project with no GitHub App connection returned the real validation error “project has no github repository bound”, surfaced verbatim. This is *not* a successful deploy — it is the validation chain correctly rejecting an unfulfillable request; the limitation is environmental (no GitHub App credentials in the evaluation environment), not an engine defect.

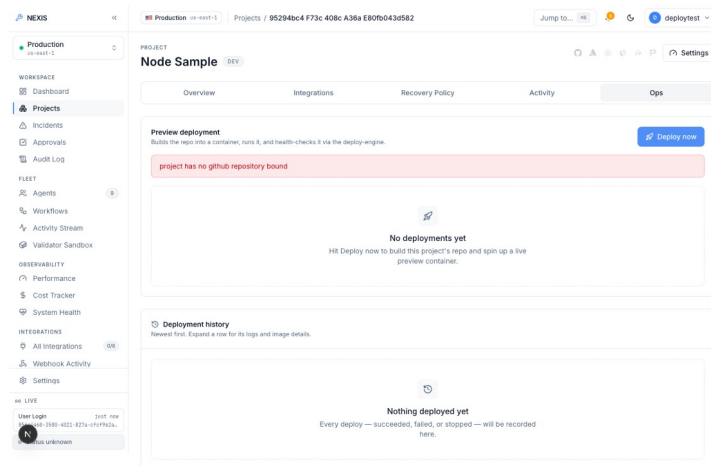


Figure 5.11: Project Ops Tab, Deploy Attempt. (a real validation error — “project has no github repository bound” — returned by the live API).

The engine was proven live independently of that limitation (Listing 5.1): `heroku/node-js-getting-started` (no Dockerfile) was cloned, detected as Node.js, given a generated Dockerfile, built, and run under resource limits; the preview URL served the app’s real rendered HTML before a clean stop and removal.

```
POST /v1/deploy (repo: heroku/node-js-getting-started, no Dockerfile present)
-> 200 OK
{"status": "running", "dockerfile_source": "generated", "detected_stack": "node",
"url": "http://localhost:54671", "image_tag": "nexis-preview-test-project:22d06076c357"}

GET http://localhost:54671/ -> real rendered HTML of the Node app (verified)

POST /v1/deploy/test-e2e-001/stop -> {"status": "stopped"}; container removed.
```

Listing 5.1: Verified live deploy-engine run.

A failed build or health check inserts a real incident with `source: "deploy_engine"` into the same Sentinel-to-RecoveryPipeline path as every other incident source.

5.2.5 Integrations and Observability

The Integrations page (Figure 5.12) manages external providers: the GitHub App (installation, repository listing, webhook-driven incidents on merged pull requests), Sentry [14], Datadog [16], and PagerDuty [15] as webhook incident sources, Slack [17] with OAuth install and one-click approve/reject from a Slack message, Argo CD, and Stripe; companion pages log webhook deliveries and probe integration health.

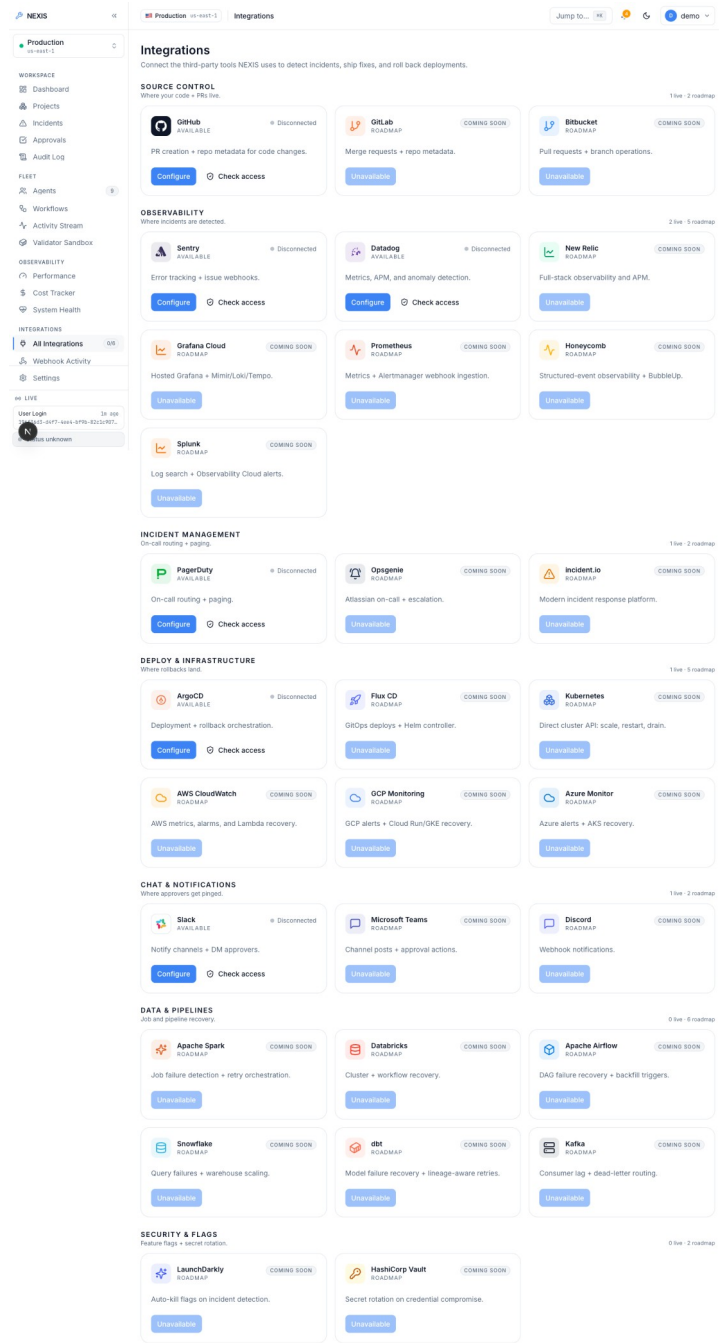


Figure 5.12: Integrations Screen. (provider catalogue: GitHub App, Sentry, Datadog, PagerDuty, Slack, Argo CD, Stripe).

Observability adds performance charts, a per-organisation token cost tracker with daily budget, parallel system-health probes (PostgreSQL, Redis, Neo4j, MinIO, Temporal), and a pgvector-backed knowledge base. The Eval Harness (Figure 5.13) runs side-by-side OpenAI-versus-Ollama comparisons across fixture incidents, with persisted transcripts, cost accounting, and administrator-only CSV export.

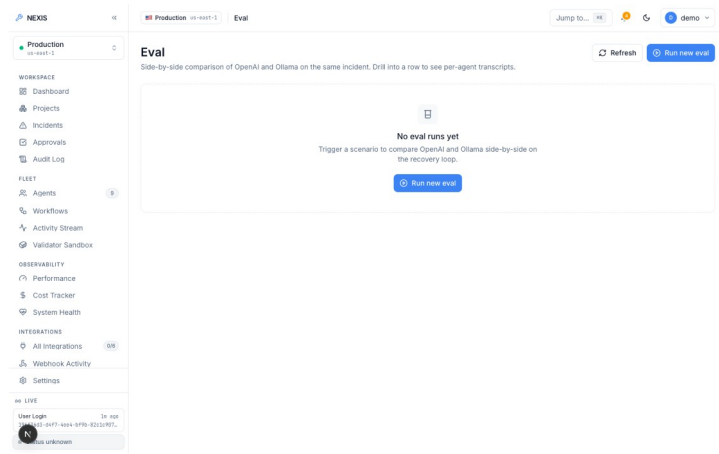


Figure 5.13: Eval Harness. (side-by-side OpenAI-versus-Ollama comparison over fixture incidents).

The Audit Log (Figure 5.14) is append-only and hash-chained — each entry incorporates its predecessor’s hash, so tampering breaks the chain and is detectable by the verify endpoint — with CSV export for offline review.

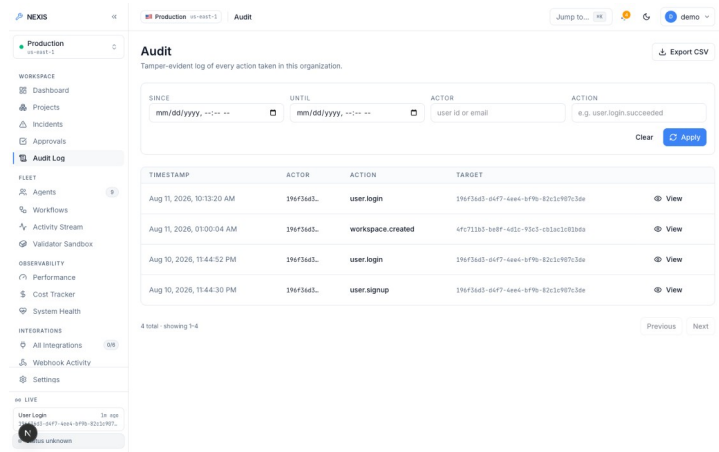


Figure 5.14: Audit Log. (append-only, hash-chained entries with CSV export).

5.3 Database Schema

State lives in PostgreSQL 17 across 33 sequential migrations. Multi-tenancy is enforced in the database itself: nearly every table carries an organisation identifier, RLS policies [7] restrict queries to the calling organisation’s rows, and a request-scoped transaction binds `app.current_org_id`, so even a query omitting an organisation `WHERE` clause cannot touch another tenant’s data. Two deliberate exceptions: `users` is global identity (tenancy applied through `org_members`), and `sessions` is user-scoped — a login session belongs to a person, not a tenant. Table 5.1 lists the most important tenant-scoped tables.

Table 5.1: Key tenant-scoped tables in the Nexis PostgreSQL schema.

Table	Purpose	RLS scope
<code>incidents_raw</code>	Every ingested fault event, from webhooks, fixtures, and the deploy engine	Organisation-scoped
<code>workflow_runs</code>	One row per Temporal RecoveryPipeline execution, with per-step activity events	Organisation-scoped
<code>approval_decisions</code>	Terminal approval-gate outcomes:	Organisation-scoped

Table	Purpose	RLS scope
	approve, reject, modify, timeout	
feedback_examples	RLHF training rows persisted from every terminal approval decision	Organisation-scoped
audit_log	Append-only, hash-chained record of privileged actions	Organisation-scoped

Neo4j holds the dependency graph the Pathfinder traverses, MinIO the encrypted patch blobs referenced from workflow_runs, and pgvector columns back knowledge-base retrieval; Temporal’s durable event history can reconstruct the per-step records in workflow_runs in a recovery scenario.

Chapter 6: Testing and Results

This chapter reports the verification evidence as it occurred: static verification of every service; live end-to-end runs driven by a real local Large Language Model (LLM) rather than stubs or mocks; and the defects uncovered.

6.1 Static Verification

Every command in Table 6.1 was executed against the final codebase — the result column is not aspirational. The Go suites include the JSON Web Token (JWT) clock-injection tests of Section 6.4 — silently failing earlier, now genuinely passing.

Table 6.1: Static verification commands and results.

Component	Command	Result
Go modules: control-plane, validator, gitops, deploy-engine	<code>go build ./... && go vet ./... && go test ./...</code>	Zero errors, zero vet findings, all tests pass
Python: causal-inference; validator Hypothesis sidecar [9]	<code>pytest</code>	38 tests passing; sidecar suite passing
apps/web — Next.js	<code>pnpm typecheck && pnpm lint && pnpm build</code>	All three gates pass

6.2 Live End-to-End Recovery Runs

Static verification establishes per-service soundness, not that the nine agents, the Temporal workflow [2], the validator sandbox, the approval gate, and the live console cooperate. Two recovery runs were therefore executed against the real system — real PostgreSQL with Row-Level Security active, real Temporal workflows, a real Neo4j graph, and a real local LLM (Ollama [20] serving `qwen2.5-coder:14b` and `gpt-oss:20b`) generating the actual plans, diffs, and tests. Nothing was mocked or replayed.

6.2.1 Run 1: Approval-Gate Timeout — the Safety Mechanism Under Real Timing Pressure

A fault fixture was injected, Sentinel detected it, and the pipeline ran to completion: the Architect produced a plan, the Backend agent a unified-diff patch, the Quality Assurance (QA) agent Hypothesis property tests, and the Validator executed the patch against them in its locked-down container sandbox — every Layer 1 and 2 step green. At the human approval gate no decision arrived within the 120-second window; the workflow resolved to a rejected terminal state.

This is a *positive* result: the central safety invariant is that no patch deploys without validation and approval and the system fails closed on ambiguity — an absent decision is refusal, never consent. It discarded a real generated patch rather than ship an unreviewed change, durably recording the timeout-rejected outcome — reported deliberately: the rejection is the stronger evidence that the gate is real.

6.2.2 Run 2: Approved and Succeeded in 1 Minute 40 Seconds

The second run took the same detection path; approval was granted within the window via the console’s decision dialog: classified Medium severity, the patch passed sandbox validation and the pipeline proceeded through deploy to a *Succeeded* state — 1 minute 40 seconds end-to-end. Two properties distinguish it from staged demonstration: the timeline records genuine per-agent token and cost figures from the live Ollama provider, and the raw JSON of the Backend agent’s activity event contains the actual unified diff written by `qwen2.5-coder:14b` (`provider="ollama"` in its metadata) — the deployed patch is the patch the model wrote, not a canned fixture.

6.3 Live Deploy-Engine Verification

The deploy-engine ran live against the public repository `heroku/node-js-getting-started`, which deliberately contains no `Dockerfile` — the engine’s hardest path. It cloned the repository, detected the stack from marker files

(`detected_stack: "node"`), generated a working Dockerfile from scratch (`dockerfile_source: "generated"`), built the image, ran it under resource limits, and proved liveness: an HTTP GET on the preview URL returned the app's real rendered HTML, and the stop endpoint removed the container (three HTTP calls, public repository — reproducible). Docker was unavailable; Podman [11] served as runtime via the same Docker-compatible API [10]. The clone was unauthenticated — the GitHub App installation-token path was not exercised, per the limitations.

6.4 Defects Found and Fixed During This Engineering Cycle

Verification surfaced two genuine defects (Table 6.2). The JWT defect was *silent* — failing without blocking anything visible — caught only because the full `go test ./...` output was read rather than skimmed; the migration defect manifests only at apply time.

Table 6.2: Defects discovered by verification.

Defect	Root Cause	Fix	Outcome
13 JWT tests silently failing	<code>exp</code> claims checked against the wall clock, not an injected test clock	Inject clock via <code>jwt.WithTimeFunc</code> (one line)	All 13 pass deterministically; production behaviour unchanged
Migration failed to apply	Reserved PostgreSQL keyword <code>current_role</code> used unquoted as a column name	Column renamed to <code>existing_role</code>	Applies cleanly; Go suites remain green

One pre-existing out-of-scope bug was deliberately left unfixed: the workspace-onboarding poller receives a 404 from `/v1/workspaces/{id}/events`, though the workspace provisions server-side; it is recorded in the limitations.

6.5 Testing Summary

Static verification is 100% clean across every service; atop it sit three live runs — two nine-agent recovery executions exercising both approval-gate branches, and one deploy-engine run — none simulated, every LLM step served by a real local model. Equally important, what was *not* tested: no formal load or performance testing was performed, no throughput or latency under multi-tenant traffic is claimed, and the 1-minute-40-second figure is one observed run, not a statistical claim. Load testing and a larger sample of recovery runs, supporting meaningful Mean Time To Recovery (MTTR) distributions, are future work (Chapter 7).

Chapter 7: Conclusion and Future Work

7.1 Overview

This project set out to close the loop that existing Artificial Intelligence (AI) developer tools leave open — from detected software failure to deployed, verified fix — with a human as a single decision point rather than the pipeline itself. The delivered platform, Nexis, realises that goal as a working system, not a design proposal.

The central deliverable is the nine-agent closed-loop self-healing pipeline, durably orchestrated by Temporal [2]: a five-agent Execution Team (Architect, Backend, QA, DevOps, Data Engineer) performs the engineering work, while the four-agent Self-Healing Loop (Sentinel, Pathfinder, Synthesiser, Validator) detects faults, localises root causes over the Neo4j dependency graph [3], sandbox-validates every candidate patch, and routes it through a severity-classified Approval Gate offering Approve, Reject, and Modify decisions. Chapter 6 exercises this pipeline live in two end-to-end recovery runs against a real local language model [20]: one timeout-rejected exactly as the safety design intends when its 120-second approval window elapsed, and one carrying a detected fault through planning, patch generation, property-based test validation, and human approval to a Succeeded deployment in approximately one minute and forty seconds.

The second major deliverable, a from-scratch Docker-compatible preview-deploy engine [10], detects a repository's stack, generates a missing Dockerfile, builds and serves the container under resource limits, and returns a live preview URL; its build and health-check failures feed the same self-healing loop, and it was proven live on a public repository lacking a Dockerfile. Around both sits the complete multi-tenant platform — SSO/MFA authentication [12], row-level RBAC [7], Stripe billing [13], GitHub, Sentry, Datadog, PagerDuty, and Slack integrations [14, 16, 15, 17], and OpenTelemetry observability [5] with a hash-chained audit log — all of it passing full build, lint, and test suites.

7.2 Limitations of the Project

Academic integrity requires that the boundary between what was demonstrated and what was not be stated plainly. The following limitations are real, known, and deliberate; none is hidden behind softer wording.

- **No production credentials were exercised.** No real GitHub App or OpenAI production credentials existed in the evaluation environment used for this report. The live evidence was produced with local Ollama models [20] and an unauthenticated clone of a public repository; a fully credentialed production path — ending in a real merged GitHub pull request — was not exercised end-to-end.
- **The platform is not yet live on its domain.** The `nexis.sh` domain is registered but the platform is not yet deployed to it. All evidence in this report comes from local, native execution. Docker was unavailable in the build environment, so Podman [11] was substituted as a runtime compatible with the Docker Application Programming Interface (API) and shown to behave identically.
- **Fine-tuning is a manual step by design.** The Reinforcement Learning from Human Feedback (RLHF) data-export pipeline is real and tested — every terminal approval decision, including engineer-edited diffs, is persisted and exportable as JSON Lines (JSONL) — but actually submitting a fine-tuning job against that export is a deliberate manual step involving real money and a real external API call, not an automated one.
- **One of twenty-seven demo scenarios is fully wired.** The Live-Demo console lists twenty-seven curated fault scenarios, of which exactly one is wired end-to-end to a real fixture and classification path today; the remaining twenty-six are User Interface (UI)-complete but backend-pending.
- **The causal ranking is a proxy, not a fitted causal model.** The causal-inference sidecar ranks root-cause candidates using an honest statistical and graph-structural scoring method with a transparent per-component confidence breakdown. It is explicitly not a fitted DoWhy structural causal model [4], because no interventional production data exists against which such a model could be fitted; claiming otherwise would be an overstatement.

- **A known unrelated bug was left unfixed.** A pre-existing bug in the workspace-onboarding progress poller (a 404 from `/v1/workspaces/{id}/events`) was found during evaluation and intentionally left unfixed as out of scope; the workspace itself provisions successfully server-side regardless.
- **No formal load testing was performed.** As noted in Chapter 6, no formal concurrent-load performance benchmark was run; the reported latencies come from single-run live executions, not from a controlled load harness.

7.3 Future Work

Each item is grounded in the existing architecture — a concrete next step, not a generic aspiration.

- **Credentialed production deployment** to the registered `nexis.sh` domain with a real GitHub App and paid OpenAI account — the only way to exercise authenticated clone, real pull-request creation, and merge-triggered redeploy end-to-end.
- **Complete the scenario library.** Wiring the remaining twenty-six Live-Demo cards to real fixtures extends fault-injection coverage across all five scenario categories.
- **Per-service Sentinel baselines.** Splitting the per-organisation EWMA baseline per service stops a noisy service masking a quiet one.
- **A fitted causal model.** Once interventional production data exists, fit a real DoWhy model [4] and evaluate it against the deliberately replaceable graph-evidence proxy.
- **Execute a fine-tuning run** on the RLHF export, measuring whether a tuned model lowers rejection and modification rates at the gate.
- **Real preview subdomains** via a Traefik-style reverse proxy, replacing `localhost` ports so previews can be shared beyond one machine.
- **Formal load benchmarking** to characterise throughput, queueing under simultaneous incidents, and Temporal worker saturation.
- **Human-subjects usability study.** The NASA-TLX instrument [18] is already integrated into the post-incident flow, leaving only recruitment and analysis.

7.4 Conclusion

The central claim is deliberately narrow and deliberately strong: a software fault can travel from automatic detection, through multi-agent diagnosis, patch generation, and sandboxed validation, past a single human approval, to a deployed fix inside one continuous, durable pipeline — demonstrated, not asserted: once the loop refused to proceed when its gate went unanswered; once it carried a real fault to approved, deployed resolution in under two minutes. Equally deliberate is what is not claimed: Section 7.2 draws the evidence boundary exactly where the evidence stops, and within it Nexis shows that human-gated, closed-loop fault recovery is buildable today.

References

1. GitHub Copilot. AI pair-programming assistant for code completion and suggestion. Available at: <https://github.com/features/copilot> (Accessed: 11 August, 2026).
2. Temporal. Durable workflow orchestration engine for fault-tolerant distributed applications. Available at: <https://temporal.io/> (Accessed: 11 August, 2026).
3. Neo4j. Native graph database for storing and querying highly connected data. Available at: <https://neo4j.com/> (Accessed: 11 August, 2026).
4. DoWhy. Causal inference library from Microsoft Research / PyWhy for modelling and estimating causal effects. Available at: <https://github.com/py-why/dowhy> (Accessed: 11 August, 2026).
5. OpenTelemetry. Open-source observability framework for traces, metrics, and logs. Available at: <https://opentelemetry.io/> (Accessed: 11 August, 2026).
6. Argo CD. Declarative GitOps continuous delivery tool for Kubernetes. Available at: <https://argo-cd.readthedocs.io/> (Accessed: 11 August, 2026).
7. PostgreSQL Row Security Policies. Official documentation on row-level security for multi-tenant data isolation. Available at: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (Accessed: 11 August, 2026).
8. OpenLineage. Open standard for data lineage collection and analysis. Available at: <https://openlineage.io/> (Accessed: 11 August, 2026).
9. Hypothesis. Property-based testing library for Python. Available at: <https://hypothesis.readthedocs.io/> (Accessed: 11 August, 2026).
10. Docker. Container platform for building, shipping, and running applications. Available at: <https://www.docker.com/> (Accessed: 11 August, 2026).
11. Podman. Daemonless, Docker-API-compatible container engine. Available at: <https://podman.io/> (Accessed: 11 August, 2026).
12. WorkOS. Enterprise-ready authentication and single sign-on platform. Available at: <https://workos.com/> (Accessed: 11 August, 2026).
13. Stripe. Online payment processing and billing infrastructure. Available at: <https://stripe.com/> (Accessed: 11 August, 2026).
14. Sentry. Application error tracking and performance monitoring platform. Available at: <https://sentry.io/> (Accessed: 11 August, 2026).
15. PagerDuty. Incident response and on-call management platform. Available at: <https://www.pagerduty.com/> (Accessed: 11 August, 2026).
16. Datadog. Cloud-scale monitoring and observability service. Available at: <https://www.datadoghq.com/> (Accessed: 11 August, 2026).
17. Slack API. Platform APIs for building Slack integrations and interactive messages. Available at: <https://api.slack.com/> (Accessed: 11 August, 2026).
18. NASA Task Load Index (NASA-TLX). Subjective workload assessment instrument developed by NASA Ames Research Center. Available at: <https://humansystems.arc.nasa.gov/groups/tlx/> (Accessed: 11 August, 2026).
19. pgvector. Open-source vector similarity search extension for PostgreSQL. Available at: <https://github.com/pgvector/pgvector> (Accessed: 11 August, 2026).
20. Ollama. Tool for running large language models locally. Available at: <https://ollama.com/> (Accessed: 11 August, 2026).